

# Production-Level Billing System Design

Complete Architecture · Flowcharts · Database ERD · Migration Roadmap

Generated: May 30, 2026

## Table of Contents

|     |                                       |    |
|-----|---------------------------------------|----|
| 1.  | Executive Summary                     | 2  |
| 2.  | System Architecture — 7 Layers        | 3  |
| 3.  | Bill Generation Flowchart             | 4  |
| 4.  | Payment Collection Flowchart          | 5  |
| 5.  | Notification Engine & Fine Engine     | 6  |
| 6.  | Maker-Checker Approval Workflow       | 7  |
| 7.  | Recurring Cycle Design                | 8  |
| 8.  | Database Schema — All Tables & Fields | 8  |
| 9.  | Database ERD Overview                 | 12 |
| 10. | Waiver & Concession Engine            | 13 |
| 11. | Student & Guardian Portal             | 13 |
| 12. | Reporting & Analytics — 14 Reports    | 14 |
| 13. | Integration Layer                     | 14 |
| 14. | Business Rules                        | 15 |
| 15. | Migration Roadmap — 4 Phases          | 16 |

## 1. Executive Summary

This document is the complete production-level design for a recurring, class-wise institutional billing system. It covers every layer of the architecture — database schema, service logic, admin UI, student portal, notification queues, fine engine, approval workflow, and migration strategy.

The design follows a **Hybrid Upgrade Path**: a new recurring billing engine is introduced while existing *fee\_masters* and *fee\_collections* tables are retained as backward-compatible ledgers, protecting all current reports, online payments, and collection logic during transition.

### Core Design Goals

- Class-wise billing profiles with Monthly, Half-Yearly, and Yearly recurring cycles
- Idempotent bill generation: cycle\_key unique constraint prevents duplicate bills
- Fee head amount snapshots frozen at issuance — profile edits never affect past bills
- Maker-checker approval workflow before any bulk generation executes
- Queue-based SMS/email dispatched only after successful database transaction commit
- Automated fine engine with configurable grace periods, aging slabs, and waiver workflow
- Student credit ledger for advance payments and carry-forward balances
- Full audit trail on every financial state change with user ID and timestamp
- Scholarship, waiver, and concession engine with category-based automatic application
- Backward compatibility — current reports and payment integrations continue working

### Weaknesses Addressed

| Weakness                      | Solution                                                            |
|-------------------------------|---------------------------------------------------------------------|
| No class-wise billing profile | New billing_profiles table with faculty/semester/batch scope        |
| No recurring cycle identity   | cycle_key format (2026-07, 2026-H1, 2026) with unique DB constraint |
| No bill amount snapshot       | student_bill_items freezes fee head amounts at issuance time        |
| Half-built installment model  | student_bills replaces dual due_date fields as source of truth      |
| SMS sent at wrong stage       | BillGenerated queue event dispatched after commit, not at payment   |
| No duplicate prevention       | Unique index on (billing_cycle_id, student_id) in student_bills     |
| No audit trail                | billing_runs + approval_status + notification_logs cover all events |
| No approval before bulk gen   | Maker-checker state machine with dry-run preview and reject/revise  |

## 2. System Architecture — 7 Layers

The system is organized into seven independently deployable layers. Data flows top-down: the Admin UI drives the Billing Engine, which triggers Collection, which fires the Notification layer.

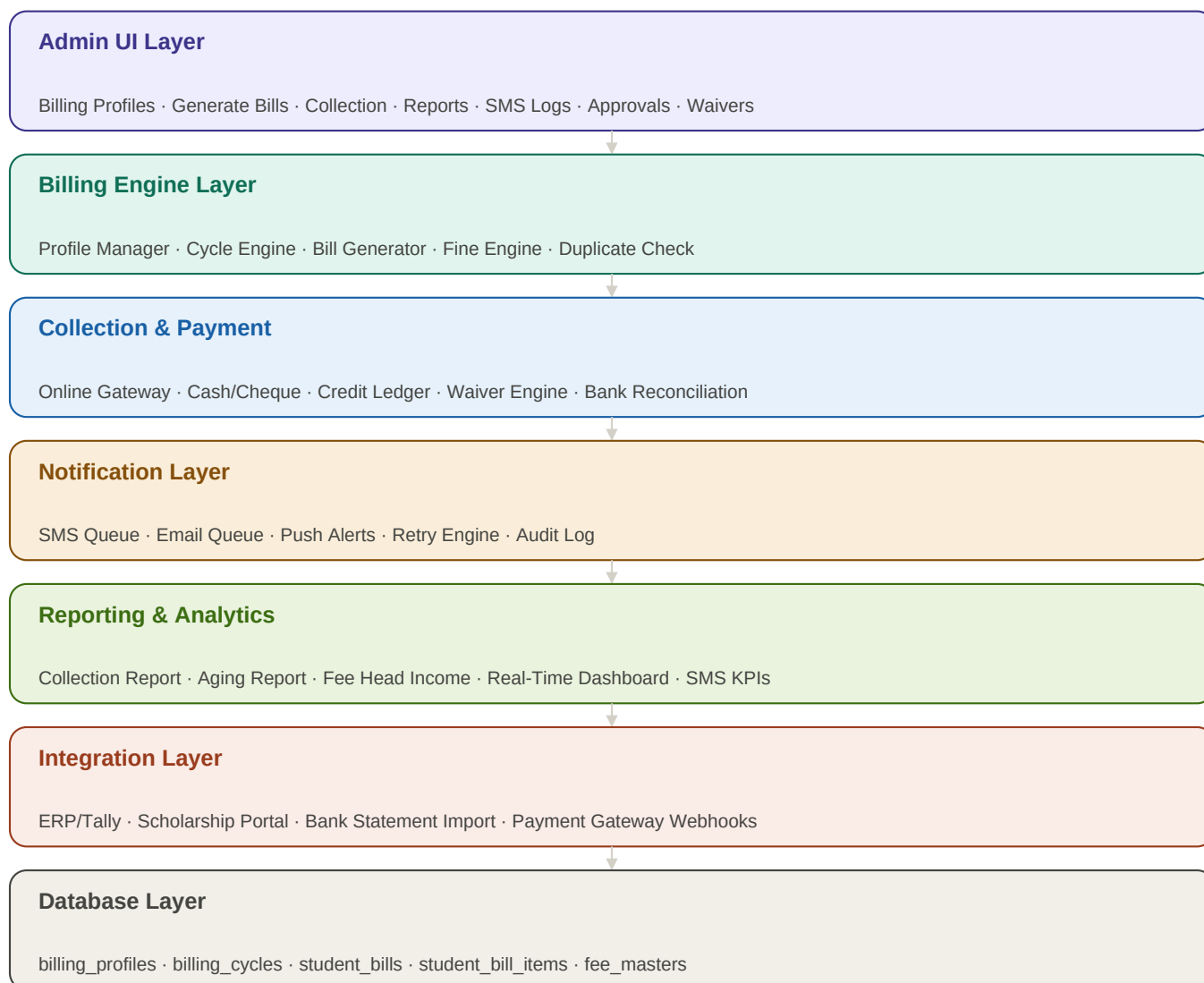


Figure 1 — 7-layer system architecture. Each layer communicates with adjacent layers through defined interfaces.

### Key Architectural Decisions

- Billing engine writes to both student\_bills (new) and fee\_masters (legacy) simultaneously during transition
- All SMS/email are dispatched to a queue — never called synchronously from controllers
- cycle\_key uniqueness enforced at DB level: unique index on (billing\_cycle\_id, student\_id)
- fee\_masters and fee\_collections remain writable through Phase 3; become read-only in Phase 4
- All approval state changes recorded in billing\_runs with initiated\_by and approved\_by user IDs
- GenerateRecurringBillsService runs entirely inside a DB transaction with rollback on failure

### 3. Bill Generation Flowchart

The complete bill generation flow runs inside a single database transaction. Every step either succeeds fully or rolls back entirely. SMS jobs are dispatched only after the commit confirms.

Bill Generation Flow — GenerateRecurringBillsService

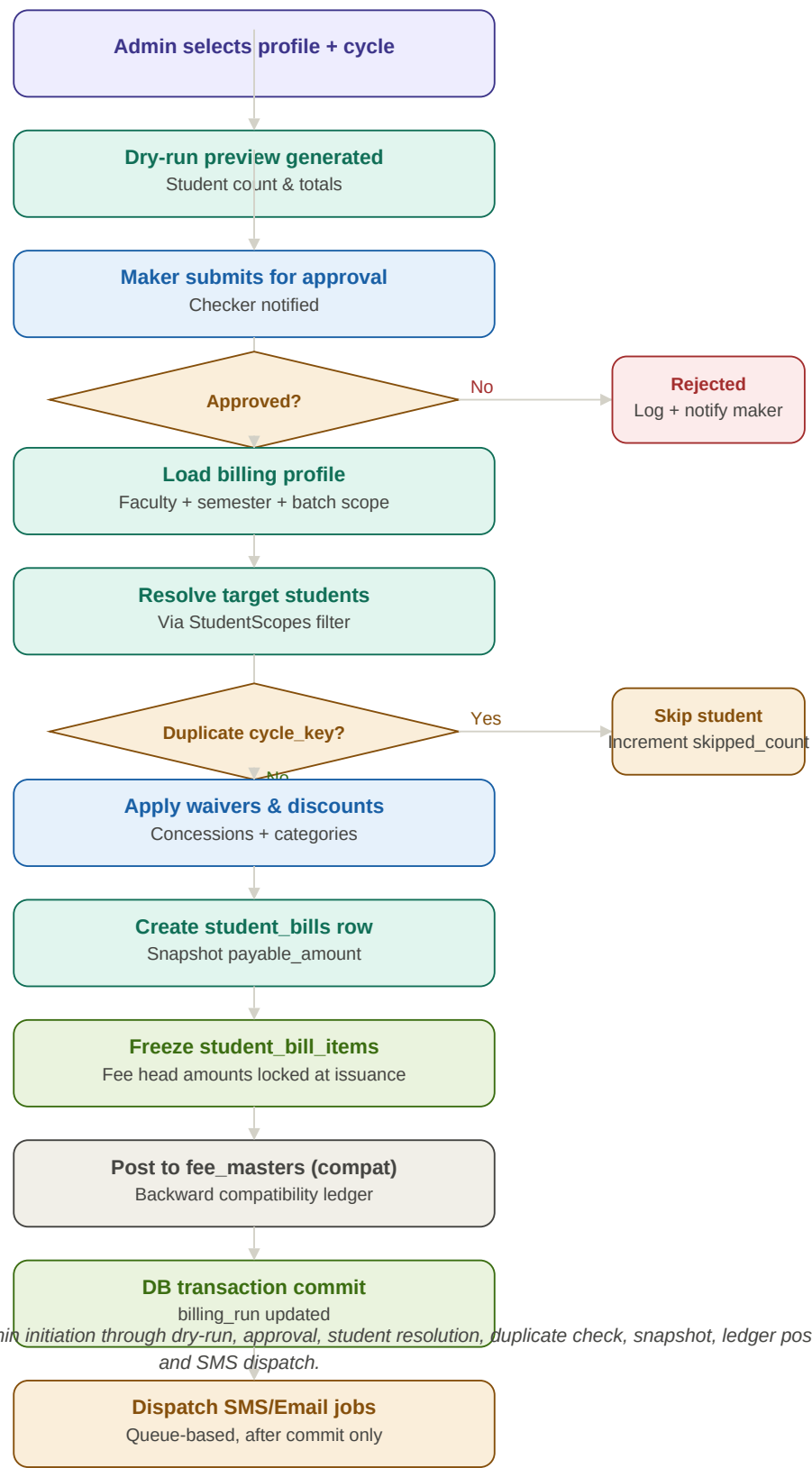


Figure 2 — Bill generation flow from admin initiation through dry-run, approval, student resolution, duplicate check, snapshot, ledger post, and SMS dispatch.

### Post-Commit Actions

- Update billing\_run: set finished\_at, status = completed, success/skipped/failed counts
- Dispatch BillGenerated job to SMS queue for each successful student\_bill
- If sms\_on\_generation = false on the profile, set sms\_status = skipped on all student\_bills
- Write generation summary to billing\_run\_audit for compliance reporting

## 4. Payment Collection Flowchart

Payments arrive through two channels — online gateway and office cashier — and converge at a common validation step before posting to the ledger.

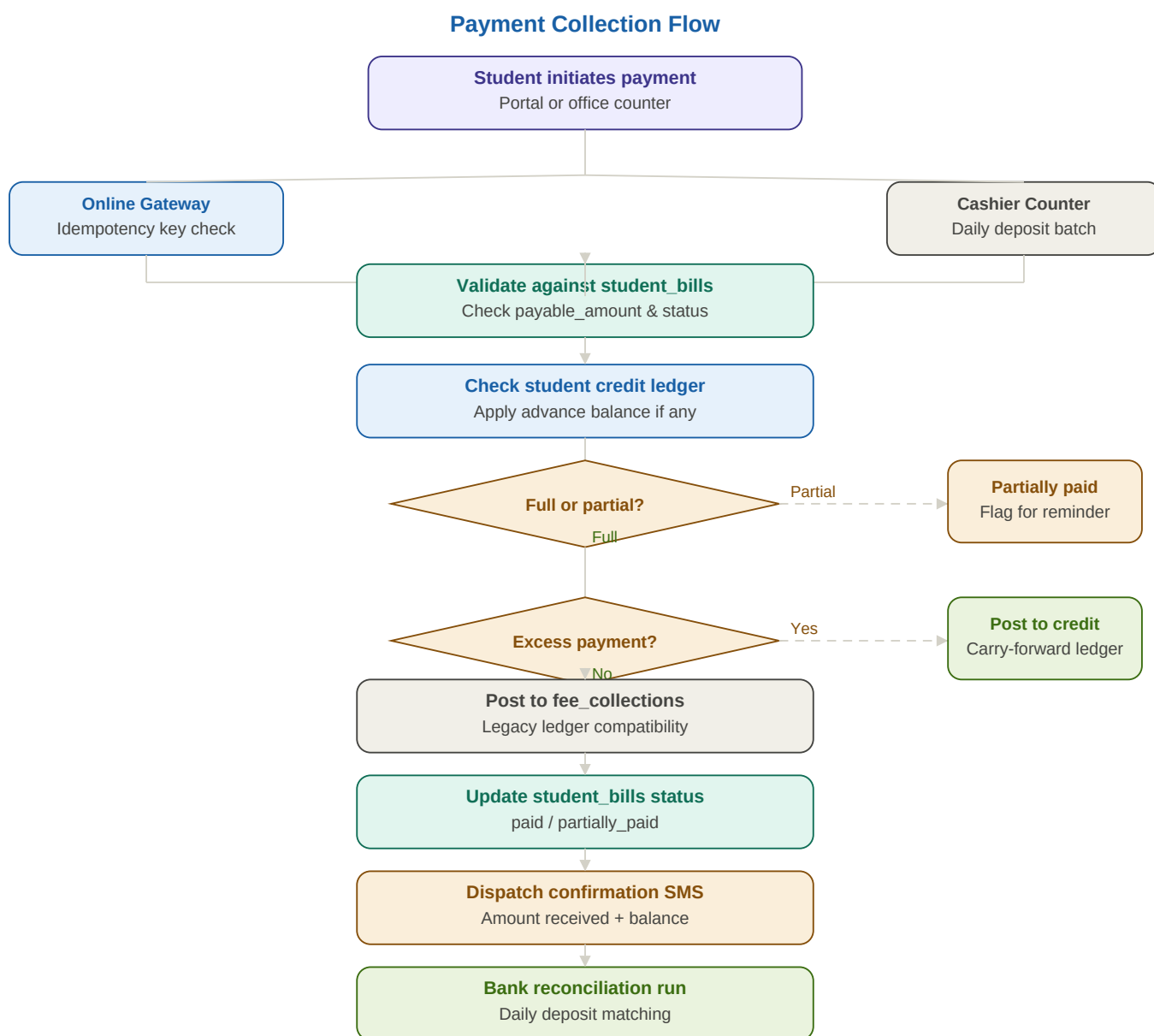


Figure 3 — Payment collection flow: online vs cash channels, partial payment handling, credit ledger application, and bank reconciliation.

### Idempotency & Reconciliation

- Every gateway webhook carries an `idempotency_key` — replayed webhooks are silently ignored
- Cashier daily deposits are imported and matched against `fee_collections` by amount + date + ref
- Unmatched deposits are flagged in the Bank Reconciliation Report for manual review
- Refunds post a negative `fee_collections` entry and credit to `student_credit_ledger`
- Refunds require checker approval before posting

## 5. Notification Engine & Fine Engine

Both engines run as scheduled daily jobs. They are independent services that read from `student_bills` and write back to it. All SMS calls go through the queue, never direct.

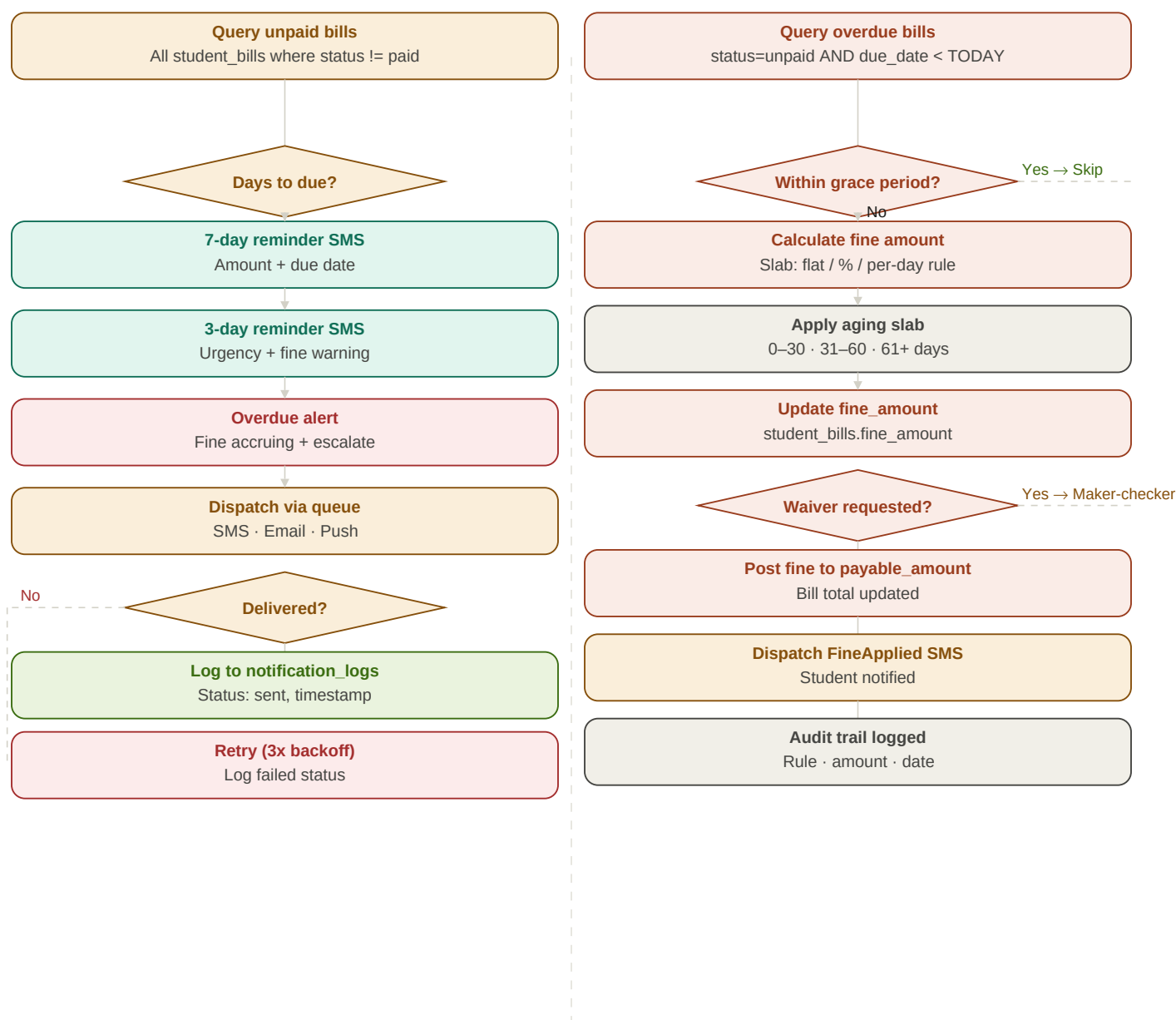


Figure 4 — Left: notification engine with 7-day, 3-day, and overdue reminders plus retry logic. Right: fine engine with grace period, aging slabs, and waiver workflow.

## Notification Events

| Event                         | Trigger                               | Channel     |
|-------------------------------|---------------------------------------|-------------|
| <code>BillGenerated</code>    | After <code>billing_run</code> commit | SMS + Email |
| <code>BillDueReminder7</code> | 7 days before <code>due_date</code>   | SMS         |
| <code>BillDueReminder3</code> | 3 days before <code>due_date</code>   | SMS + Push  |
| <code>BillOverdueAlert</code> | 1 day after <code>due_date</code>     | SMS + Email |

|                 |                            |     |
|-----------------|----------------------------|-----|
| PaymentReceived | After fee_collections post | SMS |
| FineApplied     | When fine_amount updated   | SMS |
| WaiverApproved  | After waiver approval      | SMS |



## 6. Maker-Checker Approval Workflow

The maker-checker pattern applies to bill generation, fine waivers, bulk concession grants, and manual fee adjustments. Every state change is persisted with acting user ID and timestamp.

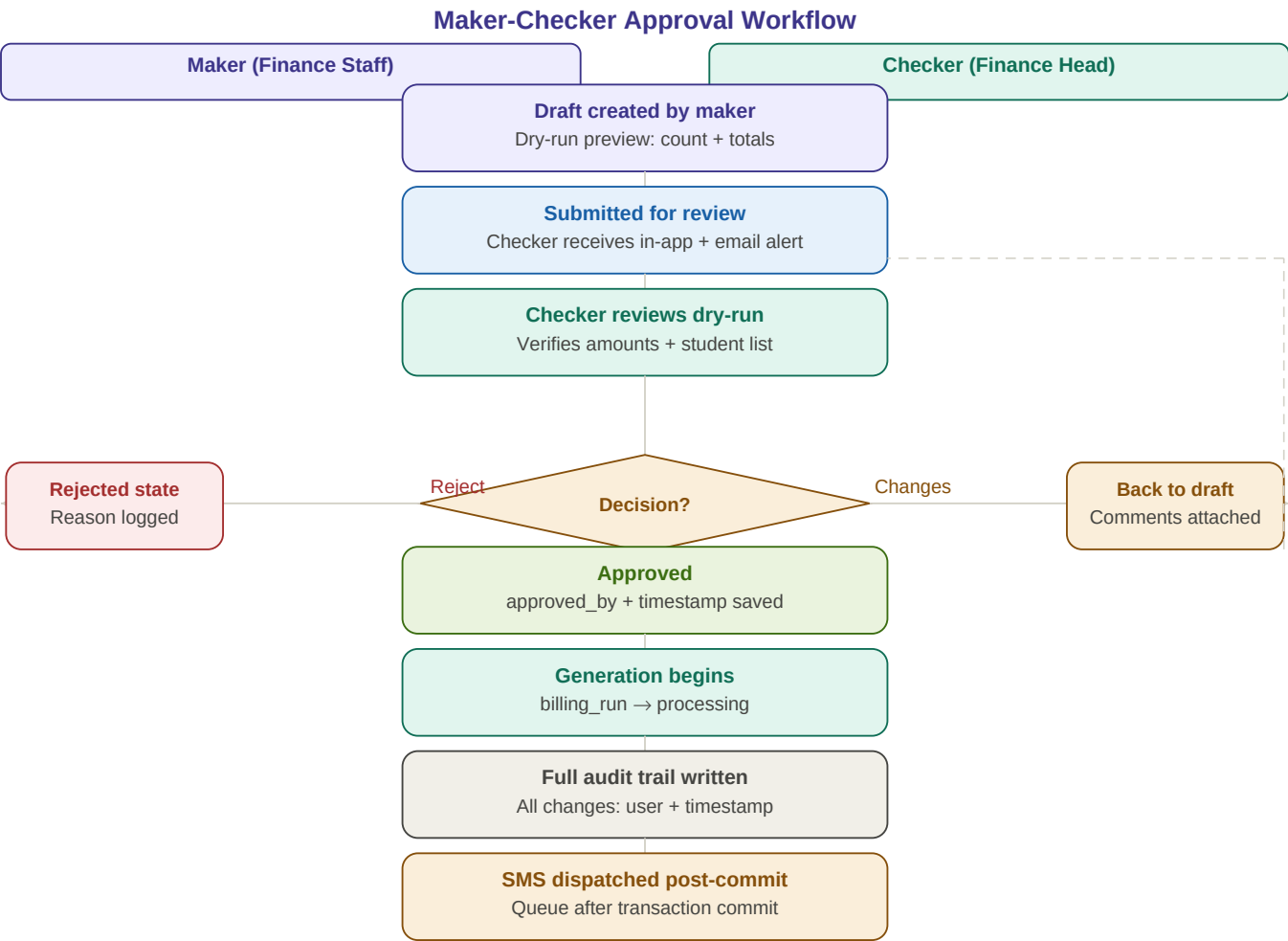


Figure 5 — Maker-checker state machine: draft → submitted → approved/rejected/changes-requested, with full audit trail on every transition.

### States & Rules

| State             | Who     | Next actions                              |
|-------------------|---------|-------------------------------------------|
| DRAFT             | Maker   | Edit, preview, submit for approval        |
| SUBMITTED         | System  | Checker notified; awaiting review         |
| UNDER REVIEW      | Checker | Approve, reject, or request changes       |
| APPROVED          | Checker | Generation begins immediately             |
| REJECTED          | Checker | Frozen; maker must create new draft       |
| CHANGES REQUESTED | Checker | Draft reopened; maker revises + resubmits |

## 7. Recurring Cycle Design

cycle\_key is the idempotency anchor. No two student\_bills rows can share the same (billing\_cycle\_id, student\_id). The key format is determined by the profile's frequency\_type.

Recurring Cycle Key Design

| Frequency   | Cycle key examples                        | Cadence       |
|-------------|-------------------------------------------|---------------|
| Monthly     | 2026-07, 2026-08 ... 2026-12, 2027-01 ... | 12 bills/year |
| Half-Yearly | 2026-H1, 2026-H2                          | 2 bills/year  |
| Yearly      | 2026, 2027                                | 1 bill/year   |
| Ad-hoc      | 2026-EXAM-FEE, 2026-HOSTEL-ADM            | As needed     |

Figure 6 — Cycle key formats by frequency type. cycle\_key is stored in both billing\_cycles and student\_bills for fast duplicate detection.

## 8. Database Schema — All Tables & Fields

Ten new tables form the core of the recurring billing engine. The existing `fee_masters` and `fee_collections` tables are extended with nullable foreign keys for backward compatibility.

### `billing_profiles` — Class-wise billing template

| Field                                | Type             | Description                            |
|--------------------------------------|------------------|----------------------------------------|
| <code>id</code>                      | uuid PK          | Primary key                            |
| <code>name</code>                    | string           | Human-readable profile name            |
| <code>academic_year_id</code>        | uuid FK          | References <code>academic_years</code> |
| <code>frequency_type</code>          | enum             | monthly   half_yearly   yearly   adhoc |
| <code>start_date</code>              | date             | Profile validity start                 |
| <code>end_date</code>                | date             | Profile validity end                   |
| <code>faculty_id</code>              | uuid FK          | Target faculty scope                   |
| <code>semester_id</code>             | uuid FK          | Target semester scope                  |
| <code>batch_id</code>                | uuid FK nullable | Optional batch filter                  |
| <code>academic_status</code>         | string nullable  | e.g. regular, ex-student               |
| <code>sms_on_generation</code>       | boolean          | Auto-send SMS after generation         |
| <code>auto_generate</code>           | boolean          | Schedule-based auto generation         |
| <code>active</code>                  | boolean          | Soft-disable without deleting          |
| <code>created_by</code>              | uuid FK          | User who created the profile           |
| <code>created_at / updated_at</code> | timestamps       | Standard audit timestamps              |

### `billing_profile_items` — Template fee heads

| Field                           | Type          | Description                              |
|---------------------------------|---------------|------------------------------------------|
| <code>id</code>                 | uuid PK       | Primary key                              |
| <code>billing_profile_id</code> | uuid FK       | References <code>billing_profiles</code> |
| <code>fee_head_id</code>        | uuid FK       | References <code>fee_heads</code> master |
| <code>amount</code>             | decimal(10,2) | Default amount for this fee head         |
| <code>is_optional</code>        | boolean       | Student can opt out of this item         |
| <code>sort_order</code>         | integer       | Display ordering on bill                 |

### `billing_cycles` — Actual recurring period

| Field                           | Type          | Description                              |
|---------------------------------|---------------|------------------------------------------|
| <code>id</code>                 | uuid PK       | Primary key                              |
| <code>billing_profile_id</code> | uuid FK       | References <code>billing_profiles</code> |
| <code>cycle_key</code>          | string UNIQUE | Idempotency key: 2026-07, 2026-H1, 2026  |
| <code>cycle_label</code>        | string        | Human label: July 2026 Tuition           |
| <code>period_start</code>       | date          | Billing period start date                |
| <code>period_end</code>         | date          | Billing period end date                  |

|                         |            |                                     |
|-------------------------|------------|-------------------------------------|
| due_date                | date       | Payment due date                    |
| status                  | enum       | draft   active   closed   cancelled |
| created_at / updated_at | timestamps | Standard audit timestamps           |

## **billing\_runs — Generation batch tracking**

| Field            | Type               | Description                             |
|------------------|--------------------|-----------------------------------------|
| id               | uuid PK            | Primary key                             |
| billing_cycle_id | uuid FK            | References billing_cycles               |
| total_students   | integer            | Total students targeted                 |
| success_count    | integer            | Bills successfully created              |
| skipped_count    | integer            | Bills skipped: already existed          |
| failed_count     | integer            | Bills that failed creation              |
| started_at       | timestamp          | When generation began                   |
| finished_at      | timestamp nullable | When generation completed               |
| initiated_by     | uuid FK            | Maker user ID                           |
| approved_by      | uuid FK nullable   | Checker user ID                         |
| approval_status  | enum               | draft   submitted   approved   rejected |
| rejection_reason | text nullable      | Checker rejection note                  |
| approved_at      | timestamp nullable | Approval timestamp                      |

## student\_bills — One bill per student per cycle

| Field            | Type                           | Description                                         |
|------------------|--------------------------------|-----------------------------------------------------|
| id               | uuid PK                        | Primary key                                         |
| billing_cycle_id | uuid FK                        | References billing_cycles                           |
| student_id       | uuid FK                        | References students                                 |
| total_amount     | decimal(10,2)                  | Gross amount before discounts                       |
| discount_amount  | decimal(10,2)                  | Applied waivers + concessions                       |
| fine_amount      | decimal(10,2)                  | Accrued late fees                                   |
| payable_amount   | decimal(10,2)                  | Net due: total - discount + fine                    |
| paid_amount      | decimal(10,2)                  | Amount collected so far                             |
| status           | enum                           | unpaid   partially_paid   paid   waived   cancelled |
| sms_status       | enum                           | pending   sent   failed   skipped                   |
| sms_sent_at      | timestamp nullable             | When SMS was dispatched                             |
| UNIQUE INDEX     | (billing_cycle_id, student_id) | Prevents duplicate bills per cycle                  |

## student\_bill\_items — Frozen fee head snapshot

| Field                   | Type             | Description                        |
|-------------------------|------------------|------------------------------------|
| id                      | uuid PK          | Primary key                        |
| student_bill_id         | uuid FK          | References student_bills           |
| fee_head_id             | uuid FK          | References fee_heads               |
| fee_head_title_snapshot | string           | Title at issuance — frozen         |
| amount                  | decimal(10,2)    | Amount at issuance — frozen        |
| source_profile_item_id  | uuid FK nullable | Original billing_profile_items row |

## student\_discounts — Waivers & concessions

| Field                  | Type             | Description                     |
|------------------------|------------------|---------------------------------|
| id                     | uuid PK          | Primary key                     |
| student_id             | uuid FK          | References students             |
| billing_profile_id     | uuid FK nullable | Scoped to a specific profile    |
| fee_head_id            | uuid FK nullable | Scoped to a specific fee head   |
| concession_category_id | uuid FK nullable | Merit/staff/EWS category        |
| discount_type          | enum             | flat   percentage               |
| amount                 | decimal(10,2)    | Flat amount or percentage value |
| reason                 | text             | Justification for the discount  |
| approved_by            | uuid FK          | Checker who approved            |
| valid_from             | date             | Discount validity start         |
| valid_to               | date nullable    | Discount validity end           |

## student\_credit\_ledger — Advance & carry-forward

| Field                      | Type             | Description                                         |
|----------------------------|------------------|-----------------------------------------------------|
| <code>id</code>            | uuid PK          | Primary key                                         |
| <code>student_id</code>    | uuid FK          | References students                                 |
| <code>credit_amount</code> | decimal(10,2)    | Available credit balance                            |
| <code>source</code>        | enum             | advance_payment   overpayment   refund   adjustment |
| <code>reference_id</code>  | uuid FK nullable | Source transaction reference                        |
| <code>notes</code>         | text nullable    | Admin notes                                         |
| <code>created_at</code>    | timestamp        | When credit was posted                              |

### notification\_logs — Full SMS/Email audit

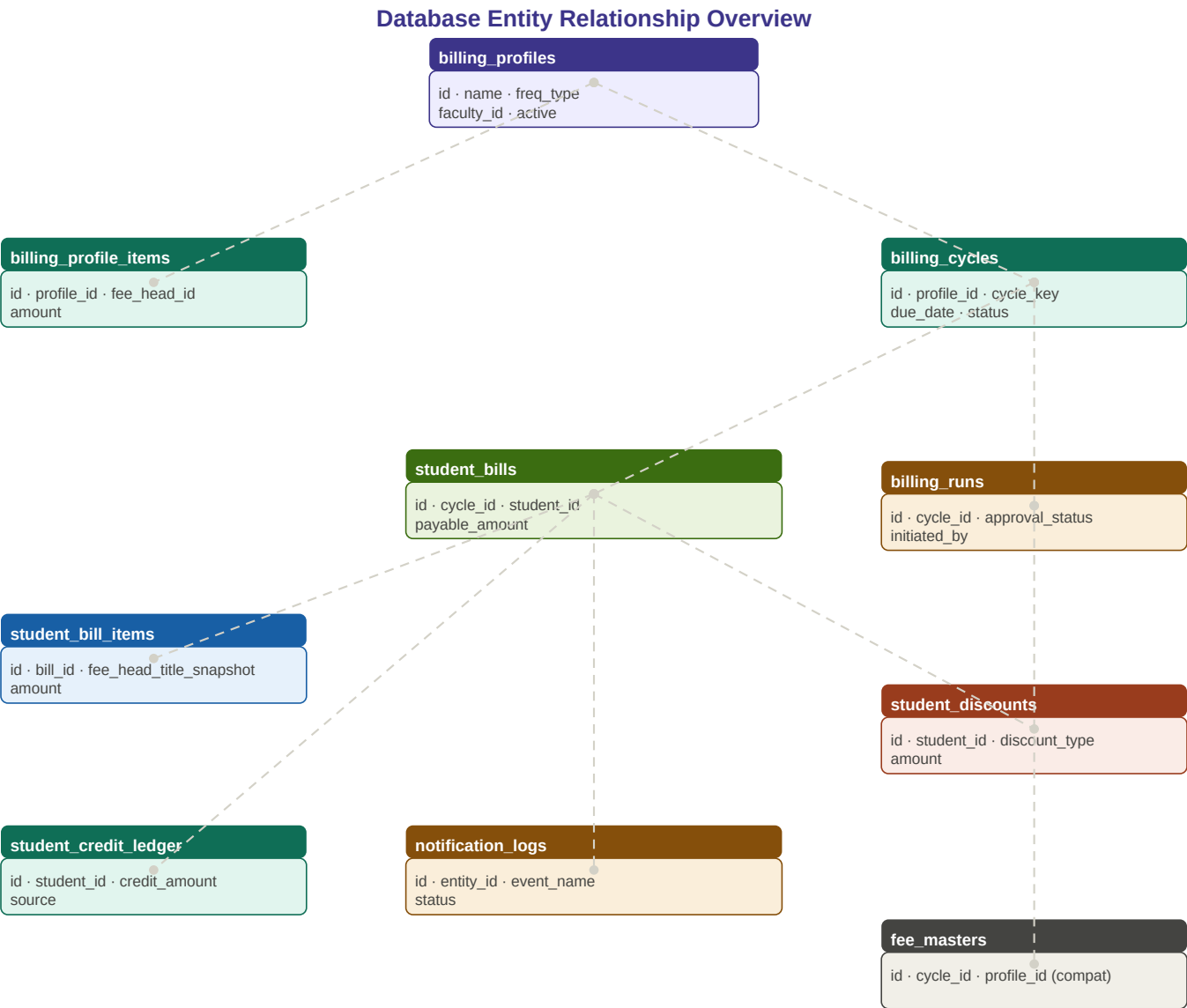
| Field                          | Type               | Description                                                     |
|--------------------------------|--------------------|-----------------------------------------------------------------|
| <code>id</code>                | uuid PK            | Primary key                                                     |
| <code>entity_type</code>       | string             | student_bill   billing_run   payment                            |
| <code>entity_id</code>         | uuid               | ID of the related entity                                        |
| <code>event_name</code>        | string             | BillGenerated   BillDueReminder   PaymentReceived   FineApplied |
| <code>recipient</code>         | string             | Phone number or email address                                   |
| <code>channel</code>           | enum               | sms   email   push                                              |
| <code>status</code>            | enum               | sent   failed   skipped   retrying                              |
| <code>attempt_count</code>     | integer            | Number of send attempts                                         |
| <code>provider_response</code> | text nullable      | Raw gateway response                                            |
| <code>sent_at</code>           | timestamp nullable | Successful delivery timestamp                                   |

### Extensions to fee\_masters (existing table)

| Field                           | Type             | Description                        |
|---------------------------------|------------------|------------------------------------|
| <code>billing_profile_id</code> | uuid FK nullable | New: link to source profile        |
| <code>billing_cycle_id</code>   | uuid FK nullable | New: link to source cycle          |
| <code>billing_run_id</code>     | uuid FK nullable | New: link to generation batch      |
| <code>cycle_key</code>          | string nullable  | New: cycle identifier for dedup    |
| <code>source_type</code>        | enum nullable    | New: recurring   manual   adhoc    |
| <code>is_recurring</code>       | boolean          | New: default false for legacy rows |

9. Database ERD Overview

The diagram below shows all 10 core tables and their foreign key relationships. Dashed lines represent FK connections. The `billing_profiles` table is the root of the hierarchy.



----- = Foreign key relationship

Figure 7 — Entity relationship overview. `billing_profiles` → `billing_cycles` → `student_bills` is the primary hierarchy. `notification_logs` is linked to `student_bills` for event tracking.

10. Waiver & Concession Engine

Concessions are applied automatically at bill generation time based on concession\_category rules. Post-generation manual waivers require maker-checker approval before updating the payable amount.

Waiver & Concession Application Flow

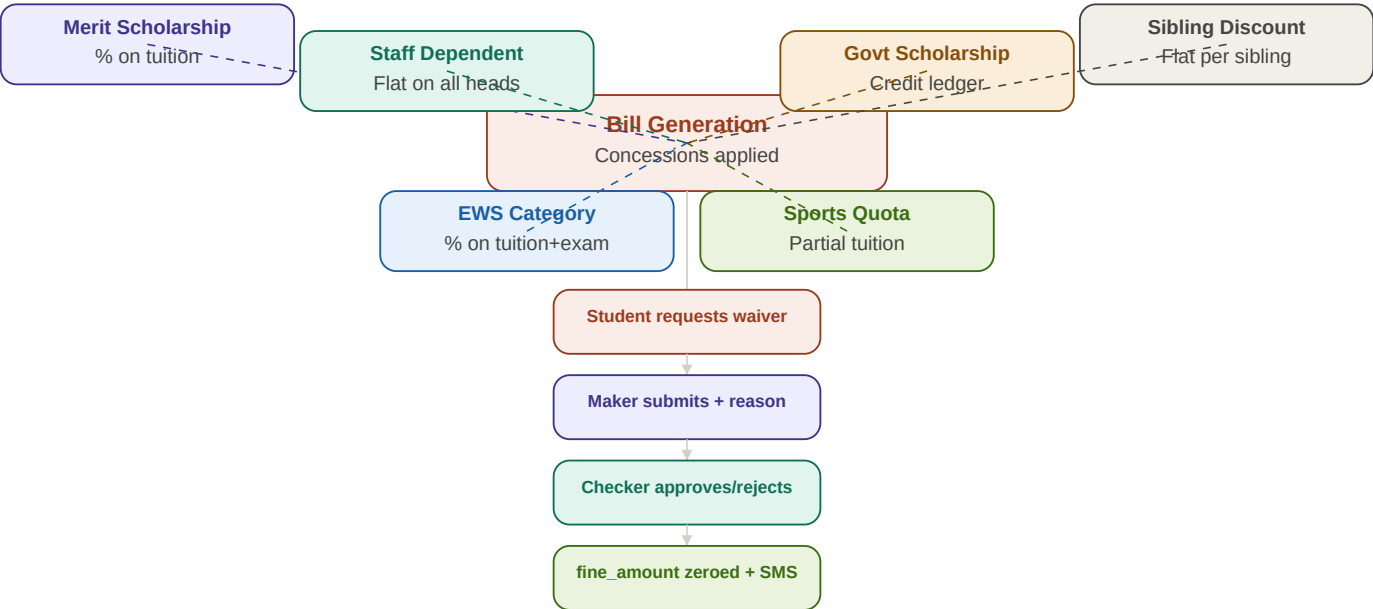


Figure 8 — Six concession categories feed into the billing engine at generation. Post-generation fine waivers follow the maker-checker approval path.

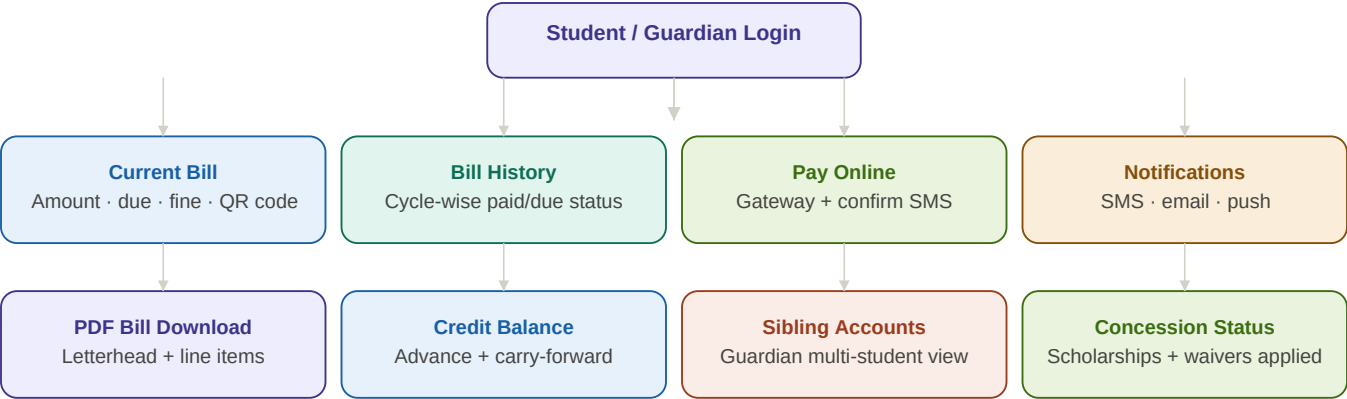
| Category          | Rule type                   | Applied on           |
|-------------------|-----------------------------|----------------------|
| Merit scholarship | % on tuition fee head       | Auto at generation   |
| Staff dependent   | Flat on all fee heads       | Auto at generation   |
| EWS category      | % on tuition + exam fee     | Auto at generation   |
| Sports quota      | Partial tuition discount    | Auto at generation   |
| Govt scholarship  | Credit posted to ledger     | Auto on next payment |
| Sibling discount  | Flat per additional sibling | Auto at generation   |



## 11. Student & Guardian Portal

Students and guardians access a dedicated portal for bill viewing, online payment, PDF downloads, and notification management. Guardians with multiple linked students manage all from one login.

Student & Guardian Portal — Screen Map



**Guardian view:** Single login shows ALL linked students with aggregated totals and individual bill drill-down

Figure 9 — Student portal screen map. Top row: primary screens. Bottom row: secondary screens. Guardian view aggregates all linked student accounts.

## 12. Reporting & Analytics — 14 Reports

The system produces 14 distinct reports covering every dimension of the billing lifecycle. A real-time dashboard provides instant visibility into collection health, SMS delivery rates, and overdue aging.

### 14 System Reports

|                                                                                |                                                                                    |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Class-wise Bill Summary</b><br>Total billed, discount, fine per class/cycle | <b>Cycle-wise Collection Report</b><br>Collection rate % and outstanding per cycle |
| <b>Student-wise Bill History</b><br>All bills + payments for one student       | <b>Fee Head Income by Cycle</b><br>Tuition vs exam vs library vs transport         |
| <b>Overdue Aging Report</b><br>0–30 · 31–60 · 61–90 · 90+ days buckets         | <b>Bank Reconciliation Report</b><br>Cashier deposits vs fee_collections match     |
| <b>SMS Delivery Report</b><br>Sent / failed / skipped by event + date          | <b>Unsent SMS Report</b><br>Bills where SMS was not delivered                      |
| <b>Failed Generation Report</b><br>billing_runs with failed_count > 0          | <b>Duplicate Prevention Log</b><br>Skipped generation attempts + cycle_key ref     |
| <b>Concession Audit Report</b><br>All discounts: category · amount · approver  | <b>Fine Collection Report</b><br>Fines applied, collected, and waived/cycle        |
| <b>Year-end Close Summary</b><br>Billed, collected, outstanding, carry-fwd     | <b>Real-time Dashboard</b><br>Live: collection rate · aging chart · KPIs           |

Figure 10 — 14 system reports organized by category. Green = collection health, Blue = operational, Red = problem detection, Amber = notification tracking.

13. Integration Layer

The billing system connects to six external systems through a hub-and-spoke integration model. All connections are asynchronous where possible to prevent gateway failures from blocking billing operations.

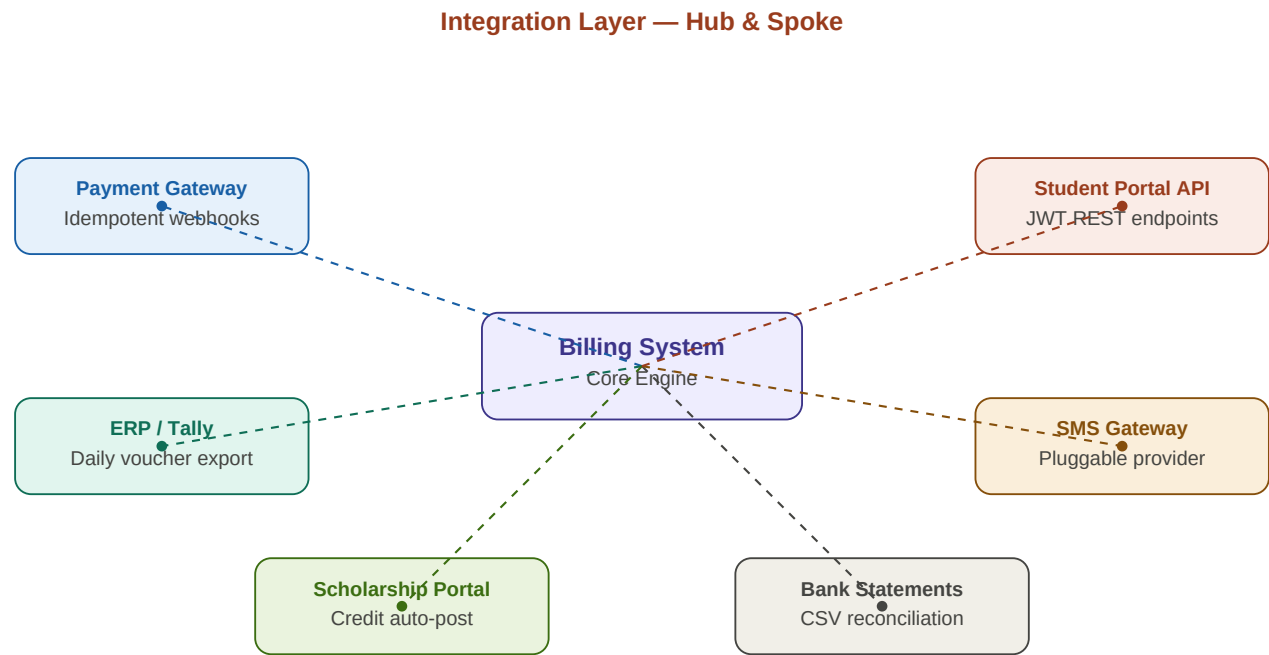


Figure 11 — Integration hub. The billing system core connects to six external systems. Dashed lines indicate asynchronous connections.

| Integration        | Direction             | Key detail                                                              |
|--------------------|-----------------------|-------------------------------------------------------------------------|
| Payment gateway    | Inbound webhook       | Idempotency key prevents double-credit on retries                       |
| ERP / Tally        | Outbound daily export | Voucher type mapped per fee head; configurable schedule                 |
| Scholarship portal | Inbound               | Scholarship amounts posted to student_credit_ledger automatically       |
| Bank statements    | Inbound CSV import    | Amount + date + ref matched to fee_collections; unmatched flagged       |
| Student portal API | Outbound REST         | JWT-authenticated; endpoints: /bills, /pay, /receipts, /credit          |
| SMS gateway        | Outbound queue        | Pluggable provider interface; all responses logged to notification_logs |

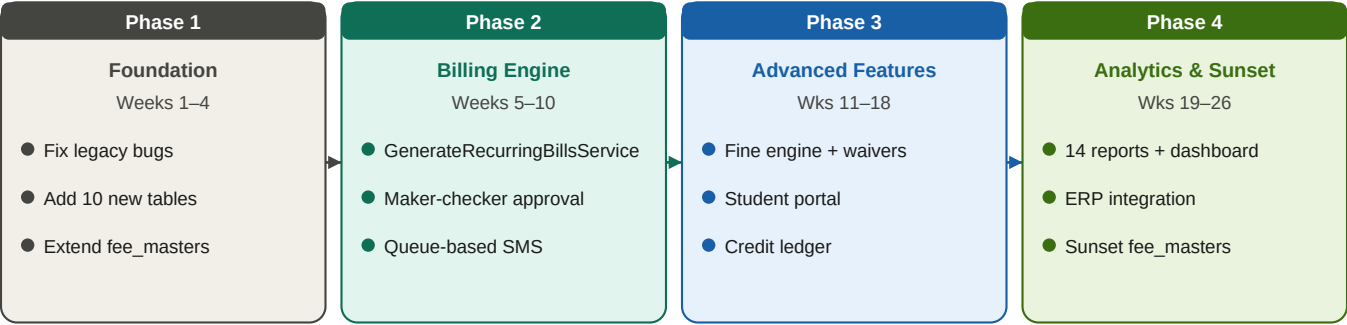
## 14. Business Rules

| <b>Class scope</b>          | Class = faculty_id + semester_id + optionally batch_id. One profile targets exactly one scope.                                   |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>One bill per cycle</b>   | A student can have at most one active bill per (billing_cycle_id, profile). Enforced by unique DB index.                         |
| <b>Amount freezing</b>      | Fee head amounts are frozen in student_bill_items at issuance. Future profile edits never affect past bills.                     |
| <b>SMS timing</b>           | SMS is dispatched only after successful DB transaction commit. Never from within the transaction.                                |
| <b>Waiver scope</b>         | Waivers are student-level overrides. Profile-level edits are never used to grant individual concessions.                         |
| <b>Fine grace period</b>    | No fine is applied until days_overdue > grace_period_days as configured per billing_profile.                                     |
| <b>Credit application</b>   | Student credit balance is applied before any fine calculation during payment posting.                                            |
| <b>Manual adjustments</b>   | Any manual change to a student_bills row requires maker-checker approval and an audit log entry.                                 |
| <b>Year-end close</b>       | Closes all billing_cycles for the year. Outstanding dues are carried forward as an adhoc cycle in the new year.                  |
| <b>Profile deactivation</b> | Deactivating a profile stops future cycle generation. Existing student_bills are not affected.                                   |
| <b>Refunds</b>              | Refunds post a negative fee_collections entry and a credit to student_credit_ledger. Require checker approval.                   |
| <b>Duplicate prevention</b> | If a billing_run is accidentally re-executed, the unique constraint on (billing_cycle_id, student_id) rejects duplicate inserts. |
| <b>Approval non-reuse</b>   | An approved billing_run cannot be re-approved. A new billing_run must be created for any re-generation.                          |

15. Migration Roadmap — 4 Phases

The migration is structured across 26 weeks into four sequential phases. Each phase delivers working, tested software — there is no big-bang cutover. Existing functionality continues working throughout.

4-Phase Migration Roadmap (26 weeks)



Each phase delivers working software — no big-bang cutover. fee\_masters becomes read-only after Phase 4.

Figure 12 — 4-phase migration timeline. Each phase builds on the previous. fee\_masters becomes read-only only after all reports are migrated in Phase 4.

**Phase 1: Foundation & Legacy Stabilization**

Weeks 1–4

- Fix BalanceFeesReminder.php email array assignment bug
- Fix missing message replacement in reminder SMS controller
- Add BillPaid SMS event after payment confirmation
- Add 10 new DB tables: billing\_profiles, billing\_profile\_items, billing\_cycles, billing\_runs, student\_bills, student\_bill\_items, student\_discounts
- Add nullable FK columns to existing fee\_masters: billing\_profile\_id, billing\_cycle\_id, billing\_run\_id, cycle\_key, is\_recurring
- Add unique index on (billing\_cycle\_id, student\_id) in student\_bills

**Phase 2: Recurring Billing Engine**

Weeks 5–10

- Build GenerateRecurringBillsService with full DB transaction support
- Implement dry-run preview with student count and total amount summary
- Implement maker-checker approval workflow with billing\_run state machine
- Add BillGenerated, BillDueReminder, BillOverdue alert events to AlertSetting.php
- Build queue-based SMS/email dispatch replacing synchronous controller calls
- Implement retry logic with exponential backoff and full notification\_logs audit
- Build admin UI: Billing Profiles, Billing Cycles, Generate Bills, Approval screens

**Phase 3: Advanced Features**

Weeks 11–18

- Build fine engine with configurable grace periods and tiered aging slabs
- Build waiver and concession engine with concession\_category management
- Build student\_discounts maker-checker approval flow
- Build student\_credit\_ledger with carry-forward and advance payment support
- Build student/guardian portal: bills, history, PDF download, online payment, sibling view
- Build bank reconciliation daily job: CSV import and amount+date+ref matching
- Build waiver approval admin screens with audit timeline

**Phase 4: Analytics, Integration & Legacy Sunset**

Weeks 19–26

- Build real-time dashboard: collection rate, aging chart, SMS delivery KPIs
- Build all 14 reports with export (PDF/Excel) support
- Build ERP/Tally daily export integration
- Build government scholarship portal inbound integration
- Migrate all online payment integrations to read from student\_bills as primary source
- Set fee\_masters to read-only — block direct writes outside compatibility layer
- Migrate all existing reports from fee\_masters to student\_bills as authoritative source
- Implement year-end close process with carry-forward adhoc cycle generation

**What Is Reused**

- StudentScopes.php — faculty, semester, batch, academic\_status filtering (used directly in GenerateRecurringBillsService)
- Fee head master module — billing\_profile\_items references the same fee\_heads table
- FeeCollection.php — payment ledger logic retained for backward compatibility
- Alert template system from AlertSetting.php — extended with new events
- Existing gateway integrations — payment webhooks continue to write to fee\_collections
- Existing collection reports — continue to work via fee\_masters compatibility layer through Phase 3

**What Is Retired After Phase 4**

- Per-student manual fee\_master creation as the primary billing engine
- fee\_due\_date2 and fee\_due\_date3 as recurring cycle storage
- Generic BalanceFeesReminder command as the notification engine
- Controller-level synchronous SMS for bulk operations
- Fallback installment logic in FeeMaster.php as source of truth
- Direct writes to fee\_masters outside the backward-compatibility layer

---

End of document — Production Billing System Design · Complete Architecture · All Flowcharts · Full Schema · Migration Roadmap